

The twice-daily loop

BBA Network — what runs on its own, where it stops, and what to reach for if you outgrow it.



Four lanes. The red lane is where the system refuses rather than guesses.

What actually runs

At 05:40 and 15:40 UTC a GitHub Actions job picks one Northern Virginia town and one trade from a fourteen-town rotation, finds businesses with a website through OpenStreetMap, drops every one already handled, audits the rest against 22 checks, ranks them, and writes the result to the run summary. Nobody is present for any of it.

The rotation is derived from the calendar rather than from stored state. A run that dies halfway and is retried repeats the same town instead of skipping ahead and quietly leaving one unvisited. Both runs in a day share a town, so the afternoon one picks up whatever the morning missed.

Where it refuses

GUARD	THE FAILURE IT PREVENTS
Already-seen ledger	Re-auditing the same forty practices every cycle and producing a full-looking shortlist containing nobody new — worse than an empty one, because it looks like progress.
Exit 3 on an empty town	Publishing an artifact and a shortlist with nothing in them. Every town runs dry eventually; that is the normal end state, not a failure.
Opt-out re-check	Emailing somebody who unsubscribed between the draft being written and the message being sent.
Unreadable store	Drafting against a stale opt-out list. "Configured but unreachable" is exactly the state where carrying on looks normal and contacts people who asked to be left alone.
Missing report	Sending a message whose body says "it's attached" with nothing attached.
Batch cap	The failure mode of a loop over a prospect list is not one bad email. It is two hundred.

Where you take over

`npm run send` writes the messages into Gmail drafts and nothing leaves the account. Sending needs `-send --yes` on top. The two outcomes are not symmetrical: a wrong draft is deleted in a second, a wrong send cannot be recalled from somebody else's inbox — so they should not be one flag apart.

If you outgrow GitHub Actions

Every link below was checked and resolves. None of them is needed today.

The honest position. You already have orchestration: a cron schedule, a job that runs steps in order, retries, secrets, logs and artifacts. At two runs a day over fourteen towns it is not the constraint. Adopting any of these before you have a paying customer adds an operational dependency to a business that has not yet been tested.

Durable job runners — the realistic upgrade

Reach for one of these when a run needs to pause for hours, fan out over hundreds of sites, or resume exactly where it failed. TypeScript-native, so they fit this codebase without a second language.

PROJECT	WHAT IT GIVES YOU THAT ACTIONS DOES NOT	WORTH IT?
Inngest github.com/inngest/inngest	Durable step functions in TypeScript. A run that fails at step four resumes at step four rather than from the top. Event-driven as well as scheduled.	When volume grows
Trigger.dev github.com/triggerdotdev/trigger.dev	Same shape, long-running jobs without timeout limits, good local development story.	When volume grows
Temporal github.com/temporalio/temporal	The serious one. Durable execution with strong guarantees, used for money movement and multi-day workflows.	Overkill here
Restate github.com/restatedev/restate	Durable execution with a lighter operational footprint than Temporal.	Overkill here

Data-pipeline schedulers

Built for dependency graphs over datasets on a schedule. All Python, which means a second language in this repository for a pipeline that is six steps long.

PROJECT	NOTE	WORTH IT?
Airflow github.com/apache/airflow	The incumbent. Heavy to operate.	No
Prefect github.com/PrefectHQ/prefect	Lighter than Airflow, same territory.	No
Dagster github.com/dagster-io/dagster	Asset-oriented; strong when data lineage matters.	No

Visual / self-hosted automation

PROJECT	NOTE	WORTH IT?
n8n github.com/n8n-io/n8n	Drag-and-drop with hundreds of connectors, Gmail included. Genuinely useful for wiring services together without code — but the audit logic already exists as tested TypeScript, and moving it into a visual editor would lose the tests.	Not for this
Windmill github.com/windmill-labs/windmill	Runs scripts as jobs with a UI on top; less rewriting than n8n.	Not yet

LLM agent frameworks

These orchestrate *models making decisions*, which is a different thing from orchestrating steps. Most of this pipeline is deterministic — discovery, the 22 checks, and qualification are rules, and rules are cheaper, faster and more auditable than a model.

PROJECT	WHERE IT WOULD GENUINELY HELP	WORTH IT?
LangGraph github.com/langchain-ai/langgraph	Stateful graphs with checkpointing. The strongest fit if you later want reply triage — reading an inbound reply and deciding whether it is interest, a question, or an opt-out.	After first replies
CrewAI github.com/crewAIInc/crewAI	Role-based multi-agent teams. Reads well in demos; the coordination overhead rarely pays for a six-step pipeline.	No
AutoGen github.com/microsoft/autogen	Conversational multi-agent research framework.	No

The two places a model actually earns its cost

- **The opening line.** Tier-1 prospects are flagged "rewrite this by hand" precisely because a generic first sentence is what gets deleted. A model that reads the site and writes one specific sentence is worth more than any scheduler on this page.
- **Reply triage.** Once mail is going out, sorting interest from questions from opt-outs — and never getting an opt-out wrong — is judgement, not rules.

Both are small, well-scoped, and need no framework to start. Neither is worth building before somebody replies.